Deploy

Serve safely within latency and cost budgets

Secure serving, reliability and cost Session 4 of 5 · 3 hours · source-aligned workshop

Outcomes you can demonstrate

- Predict 422, 429 and 503 responses.
- Trace input, output and resource controls.
- Explain why an empty metrics window cannot pass an SLO.

Three hours to harden the boundary

25 min serving/security + 20 min walkthrough

90 min labs, including a 10 min break

25 min SLO review + 20 min recap

Showcase / the service fails safely

PROVIDER FAILURE

Provider failure → private details stay private.

REPEATED REQUESTS

Repeated calls → the configured quota applies.

DATABASE FAILURE

Database failure → readiness reports unavailable.

Run the serving-failure demonstration

```
python -m training.showcase --session 4
```

Provider failures then quota: [503, 503, 429].

Invalid input: 422; failed readiness: 503.

Security, Compliance, and Responsible Operation in LLMOps

Educative lesson 11

Concept → worked example → NexaAssist application

Grounded output can still be unsafe

A source may contain private information.

A user may request an unauthorized disclosure.

A safe-looking request can consume excessive resources.

Recognize direct and indirect injection

- Direct: the user tries to override the assistant’s rules.
- Indirect: retrieved material contains hostile instructions.
- Defend at data, prompt, authorization and output boundaries.

Risk names are stable; numbering is versioned

> The lesson uses older OWASP LLM01 / LLM04 / LLM06 labels.

> OWASP 2025 maps disclosure to LLM02 and consumption to LLM10.

> Record the list version when naming a risk.

Layer the controls around generation

1. Before: validate, quota and redact the query.
2. During: bound context and output tokens.
3. After: detect PII and verify cited quotes.

Reuse the query PII rail

> scrub_request runs before embeddings and generation.

> The same pattern engine masks corpus identifiers.

> Regex misses names and addresses and can misclassify digits.

New / block detected PII before returning

```
if any(redact(text)[1] for text in (draft.answer, *(e.quote for e in draft.evidence))):
    logger.warning("answer_output_pii_blocked")
    return response
```

Check answer text and evidence quotes; return the existing safe abstention.

Preserve citation integrity when blocking

> Never rewrite a quote and still call it exact.

> On a detected identifier, discard the draft and citations.

> Log only the event; never log the sensitive text.

Practical 2 / inspect security tests

```
Run input_rail, output_cap and rate_limit tests.
```

Run test_readiness_timeout.py and the answer replay.

Explain both protected cases and known detector limits.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Resource governance has explicit bounds

- Request query length is capped.
- CHAT_MAX_OUTPUT_TOKENS bounds answer generation.
- RATE_LIMIT_PER_MINUTE is a per-process quota.

Treat residual risk as part of the result

- Patterns cover selected identifiers, not all personal data.
- Quotas are per process; proxy behavior affects client identity.
- Authorization and retention still need deployment decisions.

Inference Infrastructure: Hardware, Economics, and Latency

Educative lesson 12

Concept → worked example → NexaAssist application

Split response time into useful measurements

- TTFT: request start until the first output token.
- TPS: generation rate after output begins.
- End-to-end latency also includes queues and retrieval.

Weight memory is only the starting estimate

Approximate decimal GB = parameter billions × bytes/parameter.

Scenario: 8B × 2 bytes = 16 GB for FP16 weights.

Reserve additional memory for cache, runtime and concurrency.

Quantization changes the weight budget

FP16 uses roughly 2 bytes per parameter.

INT8 uses roughly 1; packed INT4 roughly 0.5.

Quality, kernel support and metadata overhead still matter.

The KV cache grows with workload shape

- Longer context requires more cached attention state.
- Concurrent sequences increase the live memory footprint.
- A model loading successfully does not prove capacity.

API versus self-hosting is a utilization decision

- API: variable token cost plus provider limits and governance.
- Self-hosting: fixed capacity cost plus operating effort.
- Compare representative traffic, privacy and reliability needs.

Separate input and output prices in the estimate

> Scenario: 1,000 queries/day, 500 input + 200 output tokens.

> At $0.10/$0.50 per 1M: $0.15/day, $4.50/30 days.

> These are scenario prices, not a current provider quote.

Parallelism and batching solve different constraints

- Replicas distribute requests when a model fits per worker.
- Sharding distributes a model that needs multiple devices.
- Batching trades queue delay for better resource utilization.

Practical / calculate capacity and spend

```
Run: python -m training.session4.capacity
```

Verify weight-only estimates and scenario token costs.

Write down the missing cache, runtime and concurrency inputs.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

Measure latency, errors and priced tokens

- Use /metrics/answers and app.evaluation.slo_check.
- Missing traffic or prices → not_measured.
- Process metrics do not establish a fleet-wide SLO.

Streaming needs a separate contract

> Current answers are buffered and validated as a whole.

> Total generation time does not measure TTFT or TPS.

> Streaming requires deciding when validated content may leave.

Hosting, Serving, and Scaling LLMs in Production

Educative lesson 13

Concept → worked example → NexaAssist application

Async endpoints still need non-blocking internals

- Await asynchronous provider and HTTP clients.
- Offload synchronous database/file work explicitly.
- FastAPI does not offload every utility called inside async def.

The API owns a strict contract

- Pydantic rejects invalid or extra request fields.
- Depends injects settings and request controls.
- Routers delegate; provider failures become safe 503s.

Practical 1 / exercise the real API

```
Run: python -m training.session4.serving
```

Observe health 200, readiness 503 and invalid input 422.

Two provider failures then quota: [503, 503, 429].

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

A failure should reveal enough to act

> 422 → repair the request.

> 429 + Retry-After → respect the quota.

> 503 → dependency unavailable; preserve private details.

Dependency injection defines resource ownership

- Create or cache shared clients with clear lifecycle.
- Inject application settings and replace services in tests.
- Keep domain logic callable outside HTTP routes.

Health and readiness answer different questions

> /health: this process can respond.

> /ready: the configured database ping succeeds.

> Neither endpoint proves retrieval or answer quality.

New / put a deadline on the driver ping

```
def ping_database(settings: Settings) -> None:
    """Bound the readiness probe without shortening normal query deadlines."""
    with timeout(2):
        _database(settings).command("ping")
```

The two-second driver deadline applies to readiness, not ordinary retrieval.

Package the existing application

- Read app/Dockerfile and Dockerfile.dockerignore.
- Inspect locked dependencies, non-root user and probe.
- Keep API secrets outside the image build context.

Practical 3 / inspect operational tests

```
Run tests/test_slo.py and tests/test_telemetry.py.
```

Map a request through FastAPI and workflow spans.

Explain why transport-only API tests omit lifespan.

PAIR LAB / Predict → run → inspect → explain / uv run --no-sync

A reproducible image still needs safe deployment

- Freeze dependencies and build with an excluded-secret context.
- Inject secrets at runtime; do not COPY .env into images.
- Review networking, readiness and rollout in the target platform.

Cost follows measured tokens

> Cost = (input tokens × input price + output tokens × output price) / 1M.

> Monthly estimate = priced query cost × expected query volume.

> Verify current provider prices before the live exercise.

Exit gate / can we serve it responsibly?

- Submit API failure matrix and limitation note.
- Define representative staging traffic and approved budgets.
- Hold a live release when required SLOs are unmeasured.

Next / detect, investigate and recover

- Carry the quality and operational evidence forward.
- Bring one regression and one misleading feedback item.
- Session 5 rehearses the operating loop.